

Sim Racing Button Box — Complete Master Build Guide

Version: 1.0
Date: August 2026
Hardware: Raspberry Pi Pico 2 + Freenove Terminal Breakout
Enclosure: 12mm MDF (custom-cut) with aluminium faceplate
Software: Arduino IDE, SimHub, Joystick Gremlin + vJoy + HIDHide
Target Game: Automobilista 2 (AMS2) — adaptable to any sim

Table of Contents

- 1. Introduction & Project Overview
- 2. Parts List, Tools & Materials
- 3. MDF Enclosure Construction
- 4. Component Mounting on the Faceplate
- 5. Wiring & Soldering Guide
- 6. Arduino Firmware — Pico 2 Code
- 7. SimHub Integration — LED & OLED Control
- 8. Button Virtualization Setup
- 9. Final Assembly & Testing
- 10. Troubleshooting & Reference

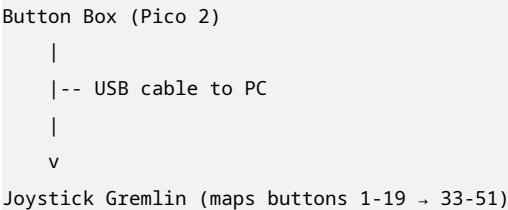
1. Introduction & Project Overview

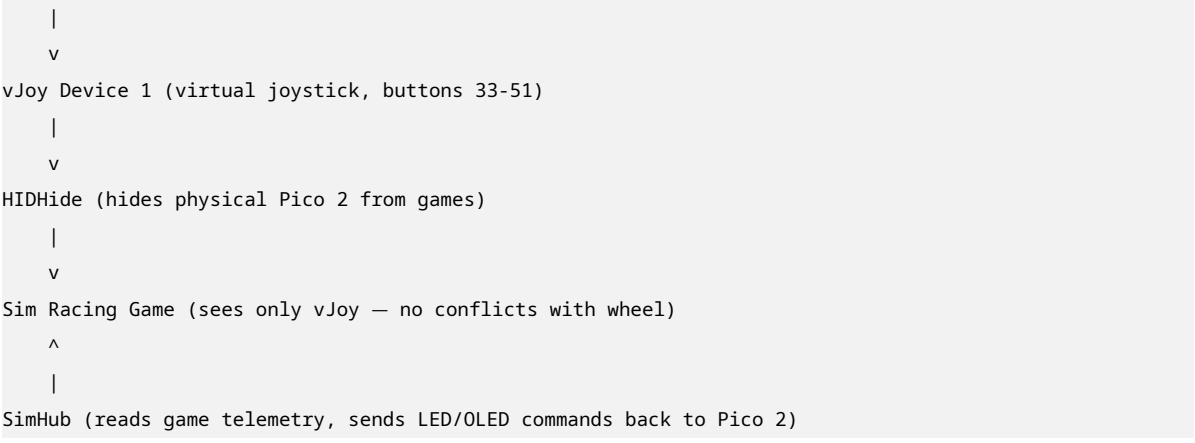
This guide documents the complete process of building a professional-grade sim racing button box from scratch. It covers every stage: designing and cutting the MDF enclosure, wiring and soldering all electronics, flashing the Raspberry Pi Pico 2 with Arduino firmware, configuring SimHub for in-game telemetry-driven LEDs and OLED display, and setting up button virtualization to avoid conflicts with your sim racing wheel.

What This Box Does

- **3× Rotary Encoders** — Adjust traction control, ABS, brake bias, etc.
- **3× Momentary Toggle Switches** — (On)-Off-(On) — map to any in-game function
- **4× Illuminated LED Pushbuttons** — Physical buttons with LEDs that flash/fade based on in-game events (low fuel, tyre wear, flags, best lap)
- **1× 2.42” OLED Screen** — Displays live telemetry: position, fuel, pit stops, tyre wear, lap times
- **25 of 26 GPIO pins used** on the Pico 2 — maximised utilisation

The Complete Signal Chain





2. Parts List, Tools & Materials

2.1 Electronic Components

Part	Qty	Specification	Notes
Raspberry Pi Pico 2	1	RP2350, 26 usable GPIO	The brains of the box
Freenove Terminal Breakout	1	Screw-terminal breakout for Pico 2	Makes wiring clean and serviceable
KY-040 Rotary Encoder	3	5-pin (CLK, DT, SW, +, GND)	With push-click function
Mini (On)-Off-(On) Toggle Switch	3	Momentary, 3-pin	Centre pin = common GND
16mm Illuminated Momentary Pushbutton	4	4-pin (2× switch, LED+, LED−)	12V LED ring, needs resistor
2.42" OLED Display	1	SSD1309, I2C, 128×64	Address 0x3C
Resistors	4	220–470Ω (330Ω ideal)	Current-limiting for LED buttons
Wire	1 reel	22AWG solid-core, multi-colour	~30cm per connection
USB Cable	1	Micro-USB or USB-C (Pico 2)	Data + power

2.2 Enclosure Materials

Part	Qty	Dimensions	Material	Notes
Faceplate	1	200mm × 150mm × 2mm	Aluminium	All components mount here
Base/Stand	1	200mm × 150mm	12mm MDF	Bottom of enclosure
Back Panel	1	200mm × 118mm	12mm MDF	Rear wall

Table 2 – continued

Part	Qty	Dimensions	Material	Notes
Top Panel	1	176mm × 75mm	12mm MDF	Angled top
Side Panel 1	1	150mm × 75mm	12mm MDF	Left side
Side Panel 2	1	150mm × 75mm	12mm MDF	Right side
MDF Sheet	1	810mm × 405mm × 12mm	MDF	Raw stock for all cuts

Note: The cutting plan (see Section 3) shows how to nest all pieces on a single 810×405mm sheet with minimal waste.

2.3 Mechanical Hardware

Part	Qty	Specification
M2 Nylon Washers	4	For OLED mounting isolation
M2/M3/M4 Screw Kit	1 set	Various lengths for assembly
Toggle Switch Nuts	3	Supplied with switches
Encoder Nuts & Washers	3	Supplied with encoders
LED Button Retaining Rings	4	Knurled rings, supplied with buttons

2.4 Tools Required

Tool	Purpose
Soldering Iron	90W adjustable digital (e.g., HANDSKIT), fine conical/chisel tip
Solder	Leaded or lead-free, 0.8mm diameter
Wire Strippers	For 22AWG wire
Flush Cutters	Snipping wire ends, resistor legs
Needle-Nose Pliers	Bending wire hooks, holding components
Small Phillips Screwdriver	For Freenove terminal screws
Drill + Bits	6mm, 7mm, 16mm for component holes
Jigsaw or CNC Router	Cutting MDF panels
Sandpaper (120-grit)	Cleaning MDF edges
Masking Tape + Marker	Labelling wires
Heat-Shrink Tubing (optional)	Insulating LED resistor joints
Multimeter (optional)	Continuity testing
Helping Hands (optional)	Holding parts steady while soldering

2.5 Software Required

Software	Version	Download Source	Purpose
Arduino IDE	2.x	arduino.cc	Flashing Pico 2 firmware
SimHub	Latest	simhubdash.com	Telemetry + LED/OLED control
vJoy	2.1.9.1	SourceForge	Virtual joystick driver
Joystick Gremlin	R15 (original)	github.com/WhiteMagic/JoystickGremlin	Button remapping
HIDHide	Latest	github.com/ViGEm/HidHide/releases	Hiding physical device
AutoHotkey	v1.1	autohotkey.com	Tray wrapper for Gremlin

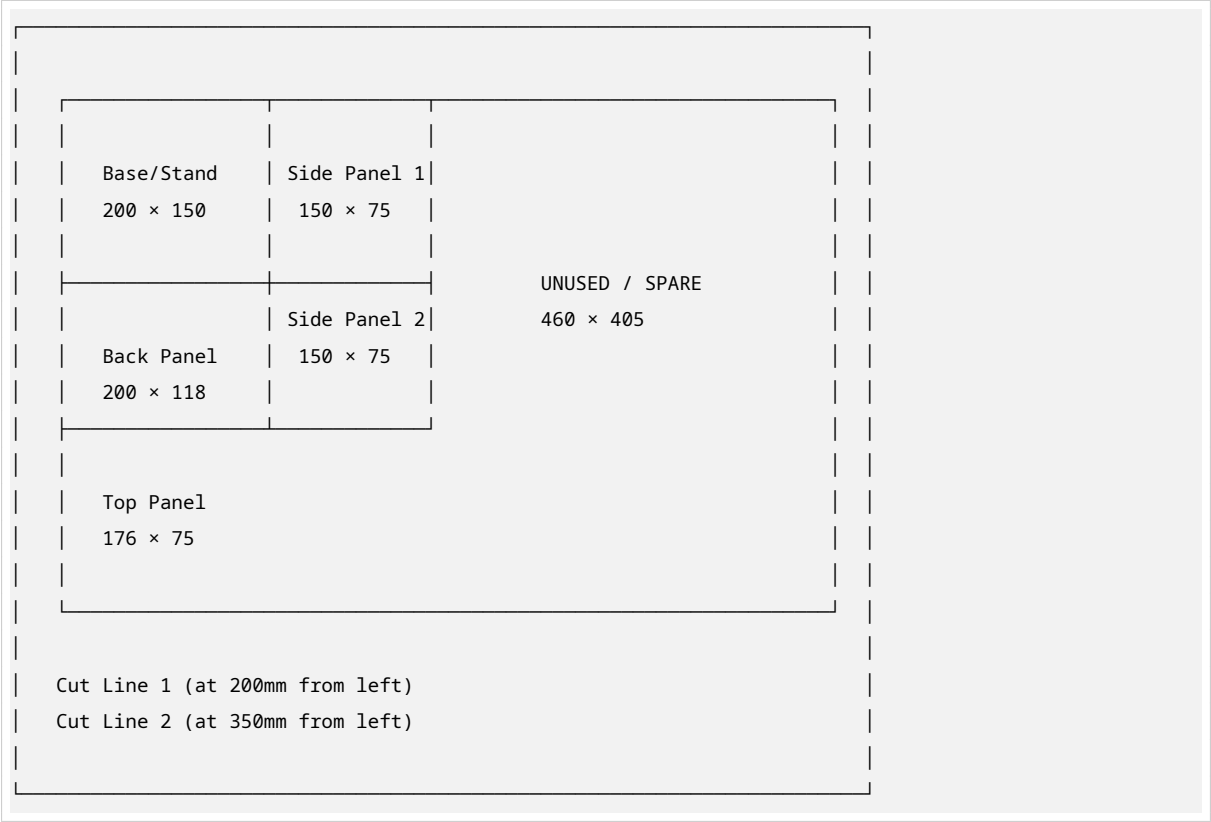
3. MDF Enclosure Construction

This section covers cutting and assembling the 12mm MDF enclosure. All pieces are designed to nest efficiently on a single 810mm × 405mm sheet.

3.1 Cutting Plan

The diagram below shows the optimal layout for cutting all MDF pieces from a single sheet. Follow these dimensions exactly to ensure all panels fit together correctly.

Sheet Size: 810mm × 405mm × 12mm



Panel Dimensions Summary:

Panel	Width	Height	Thickness	Position on Sheet
Base/Stand	200mm	150mm	12mm	Top-left
Back Panel	200mm	118mm	12mm	Middle-left
Top Panel	176mm	75mm	12mm	Bottom-left
Side Panel 1	150mm	75mm	12mm	Top-middle
Side Panel 2	150mm	75mm	12mm	Middle-middle

Tip: Label each panel with a pencil immediately after cutting. The top panel is slightly narrower (176mm vs 200mm) to allow for the angled front face.

3.2 Cutting Instructions

- Mark the sheet** using a straight edge and pencil. Mark all cut lines before cutting anything.
- First cut:** Make the vertical cut at 200mm from the left edge. This separates the left column (Base, Back, Top) from the middle column (Side Panels).
- Second cut:** Make the vertical cut at 350mm from the left edge (150mm after the first cut). This separates the middle column from the spare area.
- Horizontal cuts in left column:**
 - Cut Base/Stand (150mm from top)
 - Cut Back Panel (118mm below Base)
 - Remaining piece = Top Panel (75mm)
- Horizontal cut in middle column:**
 - Cut at 75mm from top to separate Side Panel 1 from Side Panel 2

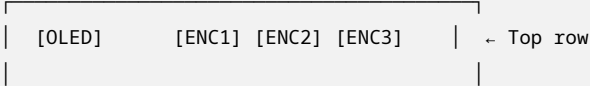
3.3 Preparing Component Holes in the Faceplate

The aluminium faceplate (200mm × 150mm × 2mm) requires precisely drilled holes for all components.

Hole Specifications:

Component	Hole Diameter	Quantity	Notes
Toggle Switches	6mm	3	Centre-punch before drilling
Rotary Encoders	7mm	3	Includes flat for anti-rotation tab
LED Buttons	16mm	4	Use a step drill for clean edges
OLED Screen	Mounting holes only	4	Use M2 screws with nylon washers

Suggested Layout (example):



[BTN1]	[BTN2]	[TOG1]	[TOG2]	← Middle row
[BTN3]	[BTN4]	[TOG3]		← Bottom row

Layout tip: Arrange components so wires route neatly toward the centre-bottom where the Pico 2 breakout board will sit. Leave at least 15mm clearance from edges for screw bosses.

3.4 Assembling the MDF Frame

The enclosure is built as an open-front box that the aluminium faceplate screws onto.

Assembly Order:

1. Attach Side Panels to Base:

- Side Panel 1 and Side Panel 2 stand vertically on the Base/Stand
- The Back Panel butts up against the rear edges of both side panels
- Use wood glue + screws (M3 or M4, 25mm length) through pre-drilled pilot holes
- Ensure all corners are square (use a carpenter's square)

2. Attach Back Panel:

- Glue and screw the Back Panel between the two side panels
- The Back Panel sits on top of the Base, not behind it
- Bottom edge of Back Panel aligns with bottom edge of Base

3. Attach Top Panel:

- The Top Panel screws to the top edges of the Side Panels and Back Panel
- It overhangs slightly at the front (the 176mm width vs 200mm base creates a lip)
- This lip helps locate the aluminium faceplate

4. Screw Bosses for Faceplate:

- Drill and insert M3 or M4 threaded inserts (or use T-nuts) into the front edges of the Base and Side Panels
- These will receive the screws that hold the aluminium faceplate
- Position: one near each corner, and one in the centre of each edge

Internal Dimensions After Assembly:

- Width: 200mm (inside = $200\text{mm} - 2 \times 12\text{mm sides} = 176\text{mm}$)
- Depth: 150mm
- Height: 150mm (Base) + 75mm (Sides) = 225mm total, but the Top Panel is angled

Note: The exact internal height depends on your chosen angle for the faceplate. A typical sim racing button box angles the faceplate at 15–30° for ergonomic reach. Adjust the Side Panel height or add a wedge to achieve your preferred angle.

3.5 Finishing

1. **Sand all edges** with 120-grit sandpaper to remove splinters and burrs
2. **Seal the MDF** with primer or clear acrylic sealer to prevent moisture absorption
3. **Paint or wrap** the exterior if desired (matte black is standard for sim racing gear)
4. **Drill a hole** in the Back Panel for the USB cable to exit (10mm diameter, with a rubber grommet to prevent chafing)

4. Component Mounting on the Faceplate

Before wiring anything, mount all components mechanically to the aluminium faceplate. This makes soldering much easier — you can work with the faceplate flat on your bench instead of trying to hold loose parts.

4.1 Mounting Order

Step 1 — Toggle Switches (easiest)

1. Insert each toggle switch through its 6mm hole from the front
2. The switch body sits behind the faceplate, the toggle lever protrudes through the front
3. Thread the supplied nut onto the threaded collar from the front and tighten firmly
4. Do NOT overtighten — aluminium can strip threads
5. Repeat for all 3 switches

Step 2 — Rotary Encoders

1. Insert each encoder through its 7mm hole from the front
2. The encoder body sits behind the faceplate, the knurled shaft protrudes through the front
3. Place the flat washer over the shaft on the front side
4. Thread the supplied nut and tighten finger-tight + $\frac{1}{4}$ turn with pliers
5. The encoder has a small metal tab (anti-rotation pin) that must sit in a shallow notch or hole drilled in the faceplate back — check this tab is seated and prevents the encoder body from rotating
6. Repeat for all 3 encoders

Step 3 — LED Pushbuttons

1. Insert each button through its 16mm hole from the front
2. The button body sits behind the faceplate
3. From the back, thread the knurled retaining ring onto the button body and tighten firmly
4. The button should not wobble or rotate when pressed
5. Repeat for all 4 buttons

Step 4 — OLED Screen

1. The OLED module has 4 corner mounting holes (typically M2 size)
2. Hold the module against the back of the faceplate where you want it
3. Mark through the mounting holes with a sharp pencil
4. Drill 2mm pilot holes through the faceplate
5. Place one M2 nylon washer on each mounting hole of the OLED PCB (this isolates the PCB traces from the metal faceplate)

6. Insert M2 screws from the front, through the faceplate, through the nylon washers, and into the OLED mounting holes
7. Tighten gently — do not crack the PCB

4.2 Component Orientation Checklist

Before proceeding to wiring, verify:

- All switches, encoders, and buttons are firmly mounted and do not rotate
- All component pins/lugs are accessible from the back of the faceplate
- There is at least 20mm clearance between the back of each component and where the Pico 2 breakout board will sit
- The OLED screen is not touching any metal that could short its exposed solder pads
- You can easily reach all pins with a soldering iron without the iron touching neighbouring plastic

4.3 Internal Layout Plan

Plan where the Pico 2 breakout board will sit inside the enclosure. Recommended position:

- **Location:** Centre-bottom of the enclosure, mounted on the Base/Stand panel
- **Orientation:** USB connector facing the Back Panel (where the cable exit hole is)
- **Mounting:** Use M2.5 or M3 standoffs (10–15mm tall) screwed into the base, with the Pico 2 breakout board screwed on top
- **Clearance:** Ensure the terminal blocks on the breakout board are accessible with a screwdriver even when the faceplate is attached

5. Wiring & Soldering Guide

This is the most critical section of the build. Poor soldering causes intermittent connections, ghost button presses, and LEDs that behave erratically. Take your time.

5.1 Critical Pre-Wiring Information

Two Things to Buy Before You Start

1. **4× resistors, 220–470Ω (330Ω is ideal).** The LED buttons require a current-limiting resistor in series with each LED. Without it, the LED can be damaged and will draw too much current from the Pico pin. Any basic resistor assortment pack works.
2. **More wire colours.** Using only one colour for ~29 separate connections is how mistakes happen. Buy a multi-colour 22AWG solid-core pack (black, yellow, green, blue, etc.). If you genuinely cannot get more colours, you **must** label every single wire end with masking tape and a marker (e.g., “GP0”, “GND”, “3V3”) the moment you cut it.

The LED Button Mistake (And Why It Matters) **What NOT to do:** Solder a wire bridging the button’s LED “—” prong straight to one of the switch prongs, so they share one ground wire back to the board.

Why it’s wrong: The LED and the switch are two completely unrelated electrical circuits that happen to live in the same plastic housing. The switch circuit tells the Pico “pressed” or “not pressed.” The LED circuit carries current to light the LED, and SimHub will switch that current on/off/flashing constantly.

If you join them at the button, the LED’s switching current travels down the same wire the Pico uses to read your finger press. That electrical noise can make the Pico think the button was pressed or released when it wasn’t — a nightmare bug to track down.

The correct approach: Every button gets **4 completely separate wires** — 2 for the switch, 2 for the LED — and none of them touch each other at the button. They only meet at the breakout board’s GND terminals, which is what “common ground” actually means.

5.2 Full Parts Inventory & Pin Count

Part	Qty	Pins per unit	Hole size
KY-040 rotary encoder	3	5 (CLK, DT, SW, +, GND)	7mm
Mini (On)-Off-(On) momentary toggle switch	3	3 (top, centre, bottom)	6mm
16mm illuminated momentary pushbutton	4	4 (2× switch, LED +, LED −)	16mm
2.42” SSD1309 OLED, I2C	1	4 (GND, VDD, SCL, SDA)	mounting holes only
Raspberry Pi Pico 2 + Freenove terminal breakout	1	26 usable GPIO	—

How full is the Pico?

The Pico 2 breaks out 26 usable GPIO pins (GP0–GP22, GP26, GP27, GP28). GP23/24/25 aren’t brought out, and VBUS/VSYS/EN/RUN/VREF are power/control pins, not GPIO.

Component	GPIO Used
Encoders: 3 × (CLK+DT+SW)	9 GPIO
Toggle switches: 3 × (up+down)	6 GPIO
OLED: SDA+SCL	2 GPIO
Buttons: 4 × (switch + LED)	8 GPIO
Total	25 of 26 GPIO
Spare	GP9

Every GND terminal on the board gets used. The single 3V3 terminal carries 4 wires twisted together (3 encoders + OLED power).

5.3 Full Pin Map (Memorise This Before Wiring)

Left terminal block (top to bottom):

Terminal	Connects to
GP0	Encoder 1 -CLK
GP1	Encoder 1 -DT
GND	Encoder 1 -GND
GP2	Encoder 1 -SW
GP3	Encoder 2 -CLK
GP4	Encoder 2 -DT
GP5	Encoder 2 -SW
GND	Encoder 2 -GND
GP6	Encoder 3 -CLK
GP7	Encoder 3 -DT
GP8	Encoder 3 -SW
GP9	<i>(spare –leave empty)</i>
GND	Encoder 3 -GND and Toggle 1 -centre pin
GP10	Toggle 1 -top pin
GP11	Toggle 1 -bottom pin
GP12	Toggle 2 -top pin
GP13	Toggle 2 -bottom pin
GND	Toggle 2 -centre pin and Toggle 3 -centre pin
GP14	Toggle 3 -top pin
GP15	Toggle 3 -bottom pin

Right terminal block (top to bottom):

Terminal	Connects to
VBUS	not used
GND	<i>(spare, unused –or backup)</i>
VSYS	not used
EN	not used – never wire anything here
3V3	Encoder 1 +, Encoder 2 +, Encoder 3 +, OLED VDD (4 wires twisted together)
VREF	not used
GP28	Button 4 -LED + (via resistor)
GND	Button 1, 2, 3, 4 -LED – (all 4 here)
GP27	Button 3 -LED + (via resistor)
GP26	Button 2 -LED + (via resistor)
RUN	not used – never wire anything here
GND	Button 1, 2, 3, 4 -switch return pin (all 4 here)
GP22	Button 1 -LED + (via resistor)
GP21	Button 4 -switch signal pin
GP20	Button 3 -switch signal pin
GP19	Button 2 -switch signal pin

Table 11 – continued

Terminal	Connects to
GND	OLED -GND
GP18	Button 1 -switch signal pin
GP17	OLED -SCL
GP16	OLED -SDA

Take a photo of this table on your phone — you’ll want it next to you at the bench.

5.4 Tools & Workspace Setup

Soldering Iron Setup:

- Fit the **smallest conical or fine chisel tip**
- Set temperature to **330–350°C**
- Before first use, and every few joints after, wipe the hot tip on the damp sponge/brass wire supplied, then immediately touch a bit of solder to the tip so it stays shiny (“tinning” the tip)
- Work on a heatproof surface, in a ventilated room
- Iron always back in its stand when not in your hand

Other tools: wire strippers, small flush cutters, small needle-nose pliers, small Phillips screwdriver (for terminal screws), masking tape + marker (for labels), helping hands clip stand (or tape parts to bench edge).

5.5 The Core Soldering Technique (Practice First)

Before touching real components, practice on a scrap of wire:

1. Strip a bit of wire, hook it around anything metal (an old screw, a paperclip)
2. Touch the iron tip to **both** the metal and the wire at the same time, holding it there
3. After about 1–2 seconds, **feed solder into the joint itself** (not onto the iron tip) — touch the solder wire to the point where iron, joint metal, and wire wrap all meet
4. The solder should melt and flow into the joint within about a second, looking wet and pulling into a shiny, smooth, slightly domed shape. Remove the iron.
5. **Do not move the joint** for 2–3 seconds while it cools. A joint disturbed while cooling turns dull, grainy and cracked — a “cold joint” that’s electrically unreliable. If yours looks dull/lumpy, just reheat it and let it cool undisturbed again.
6. Good joint = shiny, smooth, small teardrop shape. Bad joint = dull grey, cracked, or a big blob.

Key rule: Melt the solder against the joint, never against the iron tip. The iron’s job is only to heat the metal; the joint’s own heat is what should melt the solder.

5.6 Phase 1 — Toggle Switches (Easiest Joints — Do These First)

Each switch has 3 flat metal lugs with small holes (eyelets) — top, centre, bottom.

For each of the 3 switches, repeat:

1. Cut 3 wires ~30cm long. Label each end with tape now: e.g., “SW1-TOP”, “SW1-CTR”, “SW1-BOT”.

2. Strip 5mm of insulation off one end of each wire.
3. Push the bare wire through the eyelet hole in the lug and bend it into a tight hook with your pliers so it can't fall out even before soldering.
4. Heat the lug and the hooked wire together with the iron for 2–3 seconds, then feed solder in until it flows into a shiny joint. **Keep total heat time under ~4 seconds** — these switches have plastic bodies that can be damaged by prolonged heat.
5. Let it cool undisturbed, then give it a gentle tug to confirm it's solid.

Do this for the top pin, bottom pin, and centre pin of all 3 switches (9 joints total). Set the finished switches aside.

5.7 Phase 2 — LED Buttons (Corrected Method — 4 Separate Wires Each)

Each button has 4 prongs: two unmarked switch prongs, and two marked LED prongs (+ and –).

For each of the 4 buttons, repeat:

1. Cut 4 wires ~30cm long: label them now as “BTN[n]-SW-A”, “BTN[n]-SW-B”, “BTN[n]-LED+”, “BTN[n]-LED-”.
2. Strip 4mm off one end of each wire.
3. **Switch prongs (both unmarked ones):** hook a wire through each one exactly as you did with the toggle switches. Heat for no more than 2–3 seconds — these plastic housings are more heat-sensitive than the toggle switches, so work quickly and keep the iron clean and hot rather than lingering.
4. **LED – prong:** hook and solder your “LED–” wire here the same way. This wire goes straight to a GND terminal later — it is **not** connected to anything else on the button.
5. **LED + prong — this one gets a resistor.**

How to Attach the Current-Limiting Resistor Your 470Ω resistors are fine to use (within the recommended 220–470Ω range).

- **Where the resistor sits:** It goes in the middle of the LED+ path, between the button's “+” prong and the rest of the “BTN[n]-LED+” wire that runs to the board. Think of it as replacing a small section of that wire — the resistor's own two metal legs effectively become that section. It does **not** matter which leg faces the prong and which faces the wire; resistors have no direction/polarity.
- **Trimming the resistor's legs:** Straight out of the pack, a resistor's legs are long (a few cm each). Using your snips, trim each leg down to roughly **8–10mm**, leaving the resistor's coloured body intact in the middle.
- **Bend the hooks:** Using your needle-nose pliers, bend the last ~3mm of each trimmed leg into a small L-shaped hook, the same way you hook wire onto an eyelet.
- **Attach to the prong:** Hook one of the resistor's legs through/around the button's “+” prong, exactly like hooking a wire on. Heat and solder it (2–3 seconds heat, feed solder into the joint, hold still to cool).
- **Attach to the wire:** Take your “BTN[n]-LED+” wire, strip ~4mm off the end, and hook it onto the resistor's other leg the same way. Heat and solder this joint too.
- **After both joints are soldered and cooled,** check for any leftover excess: if either hook has a small tail poking out past the joint, snip that tail off close to the joint with your flush cutters.
- **Insulate it:** Slide a small piece of heat-shrink over the whole resistor if you have it (covering both joints and the body), and shrink it with the edge of the iron held near — not touching — the tubing.

If you don't have heat-shrink, wrap a small piece of electrical tape around it instead. This stops the bare resistor legs from touching the button's metal housing or the other prongs.

Repeat for all 4 buttons (5 solder joints per button — 2 switch, 1 LED—, 2 for the resistor-to-wire join — 20 joints total). Set the finished buttons aside.

5.8 Phase 3 — Rotary Encoders (Trickiest — Do These Third)

Each encoder has 5 flat gold header pins in a row: CLK, DT, SW, +, GND.

For each of the 3 encoders, repeat:

1. Cut 5 wires ~30cm long, label them “ENC[n]-CLK”, “ENC[n]-DT”, “ENC[n]-SW”, “ENC[n]-PWR”, “ENC[n]-GND”.
2. Strip only 3–4mm off each wire end — these pins sit close together, so you don't want long bare wire that could touch its neighbour.
3. Lay the bare wire flat and parallel against one pin (don't wrap it — these are flat pins, not eyelets).
4. Touch the iron to both the pin and the wire together for about 2 seconds, then feed a small amount of solder into the joint. It should flow across the gold pin quickly.
5. Hold everything still for 2 seconds while it cools.
6. If you have heat-shrink, slide a small piece over each finished joint before moving to the next pin, so neighbouring joints can't short against each other.

Repeat pin-by-pin for all 5 pins on all 3 encoders (15 joints total).

5.9 Phase 4 — OLED Screen (Delicate — Do This Last)

The screen has 4 round through-hole solder pads: GND, VDD, SCL, SDA.

1. Cut 4 wires ~30cm long, label “OLED-GND”, “OLED-VDD”, “OLED-SCL”, “OLED-SDA”.
2. Strip 5mm off one end of each.
3. Feed the bare wire through the pad's hole from the front of the board so the insulated part sticks out the back.
4. Touch the iron to the metal ring around the pad and the wire together for 2–3 seconds.
5. Feed solder onto the opposite side of the hole — it should flow around and fill the ring, gripping the wire.
6. Let it cool, then snip off any wire poking out past the joint with your flush cutters.

Repeat for all 4 pads (4 joints total). This is your most fragile component — don't rush it, but also don't linger with the iron.

Total soldering joints for the whole project: 9 (toggles) + 20 (buttons) + 15 (encoders) + 4 (OLED) = 48 joints.

5.10 Phase 5 — Wiring into the Freenove Breakout Board (Screw Terminals)

This is where you follow the **Full Pin Map** in Section 5.3, one terminal at a time. For every wire:

1. Strip 5–6mm of insulation off the board-side end.
2. Loosen the target terminal screw fully with the small screwdriver (turn counter-clockwise).
3. Insert the bare copper straight into the terminal slot.
4. Tighten the screw clockwise until firm, then give the wire a gentle tug to confirm it's locked.

Where two wires share one terminal (all the GND terminals, and the 3V3 terminal): strip both wires, twist their bare ends together by hand until snug, then insert the twisted pair together into the terminal and tighten. For the 3V3 terminal you'll be twisting 4 wires together at once — take your time lining them up before inserting.

Work through the left block first (encoders, then toggles), then the right block (3V3/power wires, then buttons, then OLED), checking off each row of your pin map table as you go.

5.11 Final Wiring Checks Before Powering On

- Visually re-check every screw terminal — nothing loose, no stray whisker of copper touching its neighbour
 - If you have a multimeter, set it to continuity/beep mode and check: each button's switch pins beep only when pressed
 - Double-check nothing is wired into **EN** or **RUN** — these are not GPIO and wiring a switch or signal into them can prevent the Pico booting
 - Confirm only 1 spare GPIO (GP9) remains unused, and every other row in your pin map has a wire seated
-

6. Arduino Firmware — Pico 2 Code

This section covers setting up the Arduino IDE for the Raspberry Pi Pico 2, installing required libraries, and flashing the complete firmware.

6.1 Arduino IDE Setup for Pico 2

1. **Download and install Arduino IDE 2.x** from [arduino.cc](https://www.arduino.cc)

2. **Add the Pico board package:**

- Go to **File** → **Preferences**
- In “Additional Boards Manager URLs”, add:

```
https://github.com/earlephilhower/arduino-pico/releases/download/global/  
↪ package_rp2040_index.json
```

- Go to **Tools** → **Board** → **Boards Manager**
- Search for “Pico” and install “Raspberry Pi Pico/RP2040” by Earle F. Philhower, III

3. **Select the board:**

- Tools → Board → Raspberry Pi RP2040 Boards → “Raspberry Pi Pico 2”

4. **Select the port:**

- Hold the **BOOTSEL** button on the Pico 2 while plugging it into USB
- It will appear as a USB mass storage device (RPI-RP2)
- Tools → Port → select the Pico 2 COM port

6.2 Required Libraries

Install these libraries via **Sketch** → **Include Library** → **Manage Libraries**:

Library	Author	Version	Purpose
Joystick	MHeironimus	2.1.1	USB HID joystick emulation
Adafruit SSD1306	Adafruit	2.5.x	OLED display driver
Adafruit GFX Library	Adafruit	1.11.x	Graphics primitives for OLED

Note: The Joystick library by MHeironimus is the standard Arduino Joystick Library. It makes the Pico 2 appear as a USB game controller to Windows.

6.3 Complete Firmware Code

Create a new sketch in Arduino IDE and paste the following code exactly:

```
// =====
// SIM RACING BUTTON BOX — Raspberry Pi Pico 2
// Reads 3 rotary encoders, 3 toggle switches, 4 LED buttons.
// Sends everything to Windows as joystick button presses (buttons 1-19).
// Also listens for simple text commands over USB so SimHub can control
// the 4 button LEDs (on/off/flash) and the OLED screen text, based on
// in-game data.
// =====

#include <Joystick.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// -----
// PIN DEFINITIONS — matches your final wiring map exactly
// -----

// Encoder 1
const int ENC1_CLK = 0;
const int ENC1_DT  = 1;
const int ENC1_SW  = 2;
// Encoder 2
const int ENC2_CLK = 3;
const int ENC2_DT  = 4;
const int ENC2_SW  = 5;
// Encoder 3
const int ENC3_CLK = 6;
const int ENC3_DT  = 7;
const int ENC3_SW  = 8;

// Toggle switches (top pin, bottom pin — centre pin is just wired to GND)
const int TOG1_TOP = 10;
const int TOG1_BOT = 11;
const int TOG2_TOP = 12;
const int TOG2_BOT = 13;
```

```

const int TOG3_TOP = 14;
const int TOG3_BOT = 15;

// LED Buttons — switch signal pins
const int BTN1_SW = 18;
const int BTN2_SW = 19;
const int BTN3_SW = 20;
const int BTN4_SW = 21;

// LED Buttons — LED control pins (through your resistors)
const int BTN1_LED = 22;
const int BTN2_LED = 26;
const int BTN3_LED = 27;
const int BTN4_LED = 28;

// Idle "off" brightness — LEDs never go fully dark, they rest at this
// faint glow for night-time button visibility.
const int LED_DIM_FLOOR = 1;

// OLED screen — confirmed working at I2C address 0x3C
const int OLED_SDA = 16;
const int OLED_SCL = 17;
#define SCREEN_ADDRESS 0x3C
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// -----
// JOYSTICK BUTTON NUMBER ASSIGNMENTS (what SimHub/games will see)
// -----
// 1 = Encoder 1 turned "up"      (e.g. Traction Control +)
// 2 = Encoder 1 turned "down"    (e.g. Traction Control -)
// 3 = Encoder 1 push-click       (e.g. TC off / reset)
// 4 = Encoder 2 turned "up"      (e.g. ABS +)
// 5 = Encoder 2 turned "down"    (e.g. ABS -)
// 6 = Encoder 2 push-click
// 7 = Encoder 3 turned "up"      (e.g. Brake Bias +)
// 8 = Encoder 3 turned "down"    (e.g. Brake Bias -)
// 9 = Encoder 3 push-click
// 10 = Toggle 1 up
// 11 = Toggle 1 down
// 12 = Toggle 2 up
// 13 = Toggle 2 down
// 14 = Toggle 3 up
// 15 = Toggle 3 down
// 16 = Button 1
// 17 = Button 2
// 18 = Button 3
// 19 = Button 4

```



```

// -----
// DEBOUNCE / TIMING SETTINGS
// -----
const unsigned long DEBOUNCE_MS = 15;    // ignore switch noise shorter than this
const unsigned long ENCODER_PULSE_MS = 60; // how long an encoder "click" button stays pressed

// -----
// STATE TRACKING
// -----

// Toggle + button switches: simple debounced digital reads
struct DebouncedInput {
    int pin;
    int joystickButton;
    bool lastStableState;
    bool lastRawState;
    unsigned long lastChangeTime;
};

DebouncedInput inputs[] = {
    { TOG1_TOP, 10, false, false, 0 },
    { TOG1_BOT, 11, false, false, 0 },
    { TOG2_TOP, 12, false, false, 0 },
    { TOG2_BOT, 13, false, false, 0 },
    { TOG3_TOP, 14, false, false, 0 },
    { TOG3_BOT, 15, false, false, 0 },
    { BTN1_SW, 16, false, false, 0 },
    { BTN2_SW, 17, false, false, 0 },
    { BTN3_SW, 18, false, false, 0 },
    { BTN4_SW, 19, false, false, 0 },
    { ENC1_SW, 3, false, false, 0 },
    { ENC2_SW, 6, false, false, 0 },
    { ENC3_SW, 9, false, false, 0 },
};

const int NUM_INPUTS = sizeof(inputs) / sizeof(inputs[0]);

// Encoders: track the combined CLK+DT state as a 2-bit value
struct EncoderState {
    int clkPin;
    int dtPin;
    int buttonUp;
    int buttonDown;
    uint8_t stateHistory;
    int8_t accumulator;
    unsigned long upReleaseTime;
    unsigned long downReleaseTime;
};

EncoderState encoders[] = {
    { ENC1_CLK, ENC1_DT, 1, 2, 0, 0, 0, 0 },
    { ENC2_CLK, ENC2_DT, 4, 5, 0, 0, 0, 0 },
};

```

```

    { ENC3_CLK, ENC3_DT, 7, 8, 0, 0, 0, 0 },
};

const int NUM_ENCODERS = sizeof(encoders) / sizeof(encoders[0]);

// Quadrature decode table — standard, well-proven
const int8_t QUAD_TABLE[16] = {
    0, -1, 1, 0,
    1, 0, 0, -1,
    -1, 0, 0, 1,
    0, 1, -1, 0
};

// LED state for each of the 4 button LEDs
struct LedState {
    int pin;
    bool on;
    bool flashing;
    unsigned long flashIntervalMs;
    unsigned long lastToggleTime;
    bool flashPhase;

    bool fading;
    bool fadeMode;
    int fadeTarget;
    int fadeCurrent;
    int fadeStep;
    unsigned long lastFadeUpdate;
};

LedState leds[] = {
    { BTN1_LED, false, false, 300, 0, false, false, false, 0, 0, 4, 0 },
    { BTN2_LED, false, false, 300, 0, false, false, false, 0, 0, 4, 0 },
    { BTN3_LED, false, false, 300, 0, false, false, false, 0, 0, 4, 0 },
    { BTN4_LED, false, false, 300, 0, false, false, false, 0, 0, 4, 0 },
};

const int NUM_LEDS = sizeof(leds) / sizeof(leds[0]);

// -----
// SETUP
// -----

void setup() {
    // All switch/button pins use the Pico's internal pull-up resistor.
    // Not pressed = reads HIGH, pressed = reads LOW (pulled to GND).
    for (int i = 0; i < NUM_INPUTS; i++) {
        pinMode(inputs[i].pin, INPUT_PULLUP);
    }

    for (int i = 0; i < NUM_ENCODERS; i++) {
        pinMode(encoders[i].clkPin, INPUT_PULLUP);
        pinMode(encoders[i].dtPin, INPUT_PULLUP);
        int clk = digitalRead(encoders[i].clkPin);
        int dt = digitalRead(encoders[i].dtPin);
    }
}

```

```

    encoders[i].stateHistory = (clk << 1) | dt;
    encoders[i].accumulator = 0;
}
for (int i = 0; i < NUM_LEDS; i++) {
    pinMode(leds[i].pin, OUTPUT);
    analogWrite(leds[i].pin, LED_DIM_FLOOR);
}

// OLED setup
Wire.setSDA(OLED_SDA);
Wire.setSCL(OLED_SCL);
Wire.begin();
display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS);
display.clearDisplay();
display.display();
// Screen stays blank until SimHub sends a DISP| command.

Serial.begin(115200); // USB serial link for SimHub LED + screen commands
Joystick.begin();     // starts the USB joystick device
}

// -----
// MAIN LOOP
// -----
void loop() {
    unsigned long now = millis();

    readDebouncedInputs(now);
    readEncoders(now);
    readSerialCommands();
    updateLeds(now);
}

// -----
// Reads all toggle switches, LED buttons, and encoder push-switches.
// Applies simple debouncing: a change is only accepted once the new
// state has held steady for DEBOUNCE_MS.
// -----
void readDebouncedInputs(unsigned long now) {
    for (int i = 0; i < NUM_INPUTS; i++) {
        bool rawPressed = (digitalRead(inputs[i].pin) == LOW); // LOW = pressed

        if (rawPressed != inputs[i].lastRawState) {
            inputs[i].lastRawState = rawPressed;
            inputs[i].lastChangeTime = now;
        }

        if ((now - inputs[i].lastChangeTime) > DEBOUNCE_MS) {
            if (rawPressed != inputs[i].lastStableState) {
                inputs[i].lastStableState = rawPressed;
                Joystick.button(inputs[i].joystickButton, rawPressed);
            }
        }
    }
}

```

```

    }
}
}

// -----
// Reads all 3 rotary encoders using full quadrature state tracking.
// A full detent click only fires once 4 valid steps in the same
// direction have been seen in a row.
// -----
void readEncoders(unsigned long now) {
    for (int i = 0; i < NUM_ENCODERS; i++) {
        int clk = digitalRead(encoders[i].clkPin);
        int dt = digitalRead(encoders[i].dtPin);
        uint8_t newState = (clk << 1) | dt;

        uint8_t lookup = ((encoders[i].stateHistory & 0x03) << 2) | newState;
        int8_t movement = QUAD_TABLE[lookup];

        if (movement != 0) {
            encoders[i].accumulator += movement;

            if (encoders[i].accumulator >= 4) {
                Joystick.button(encoders[i].buttonUp, true);
                encoders[i].upReleaseTime = now + ENCODER_PULSE_MS;
                encoders[i].accumulator = 0;
            } else if (encoders[i].accumulator <= -4) {
                Joystick.button(encoders[i].buttonDown, true);
                encoders[i].downReleaseTime = now + ENCODER_PULSE_MS;
                encoders[i].accumulator = 0;
            }
        }

        encoders[i].stateHistory = newState;

        // Auto-release the pulse buttons after their hold time expires
        if (encoders[i].upReleaseTime != 0 && now >= encoders[i].upReleaseTime) {
            Joystick.button(encoders[i].buttonUp, false);
            encoders[i].upReleaseTime = 0;
        }
        if (encoders[i].downReleaseTime != 0 && now >= encoders[i].downReleaseTime) {
            Joystick.button(encoders[i].buttonDown, false);
            encoders[i].downReleaseTime = 0;
        }
    }
}

// -----
// Listens on USB serial for simple text commands to control the LEDs.
// This is what SimHub will send.
//

```

```

// Command format, one line at a time:
//  LED1_ON           harsh instant on
//  LED1_OFF          harsh instant off
//  LED1_FLASH_300    harsh blink, 300ms on/off interval
//  LED1_FADE_ON      smooth fade up to fully on
//  LED1_FADE_OFF      smooth fade down to fully off
//  LED1_FADE_FLASH_500 smooth breathing fade, 500ms per half-cycle
//
// OLED screen command:
//  DISP|Line1 text|Line2 text|Line3 text
// -----
void readSerialCommands() {
    if (!Serial.available()) return;

    String line = Serial.readStringUntil('\n');
    line.trim();
    if (line.length() == 0) return;

    if (line.startsWith("DISP|")) {
        updateDisplay(line);
        return;
    }

    for (int i = 0; i < NUM_LEDS; i++) {
        String prefix = "LED" + String(i + 1);
        if (!line.startsWith(prefix)) continue;

        if (line == prefix + "_ON") {
            leds[i].on = true;
            leds[i].flashing = false;
            leds[i].fading = false;
            digitalWrite(leds[i].pin, HIGH);
        } else if (line == prefix + "_OFF") {
            leds[i].on = false;
            leds[i].flashing = false;
            leds[i].fading = false;
            analogWrite(leds[i].pin, LED_DIM_FLOOR);
        } else if (line.startsWith(prefix + "_FLASH_")) {
            String msPart = line.substring((prefix + "_FLASH_").length());
            unsigned long interval = msPart.toInt();
            if (interval < 30) interval = 30;
            leds[i].flashing = true;
            leds[i].fading = false;
            leds[i].flashIntervalMs = interval;
            leds[i].lastToggleTime = millis();
            leds[i].flashPhase = true;
            digitalWrite(leds[i].pin, HIGH);
        } else if (line == prefix + "_FADE_ON") {
            leds[i].flashing = false;
            leds[i].fading = true;
            leds[i].fadeMode = false;

```

```

        leds[i].fadeTarget = 255;
        leds[i].lastFadeUpdate = millis();
    } else if (line == prefix + "_FADE_OFF") {
        leds[i].flashing = false;
        leds[i].fading = true;
        leds[i].fadeMode = false;
        leds[i].fadeTarget = LED_DIM_FLOOR;
        leds[i].lastFadeUpdate = millis();
    } else if (line.startsWith(prefix + "_FADE_FLASH_")) {
        String msPart = line.substring((prefix + "_FADE_FLASH_").length());
        unsigned long halfCycle = msPart.toInt();
        if (halfCycle < 50) halfCycle = 50;
        leds[i].flashing = false;
        leds[i].fading = true;
        leds[i].fadeMode = true;
        leds[i].fadeTarget = 255;
        leds[i].fadeStep = max(1, (int)(255 / (halfCycle / 10)));
        leds[i].lastFadeUpdate = millis();
    }
}
}

// -----
// Splits a DISP|Line1|Line2|Line3 command and draws each part on the OLED
// -----
void updateDisplay(String line) {
    String remaining = line.substring(5);

    display.clearDisplay();
    display.setTextSize(2);
    display.setTextColor(SSD1306_WHITE);
    display.setCursor(0, 0);

    int lineNum = 0;
    while (remaining.length() > 0 && lineNum < 4) {
        int barIndex = remaining.indexOf('|');
        String thisLine;

        if (barIndex == -1) {
            thisLine = remaining;
            remaining = "";
        } else {
            thisLine = remaining.substring(0, barIndex);
            remaining = remaining.substring(barIndex + 1);
        }

        display.println(thisLine);
        lineNum++;
    }

    display.display();

```

```

}

// -----
// Handles harsh flashing and smooth fade modes for all LEDs
// -----

void updateLeds(unsigned long now) {
    for (int i = 0; i < NUM_LEDS; i++) {

        // Harsh flash handling
        if (leds[i].flashing) {
            if (now - leds[i].lastToggleTime >= leds[i].flashIntervalMs) {
                leds[i].flashPhase = !leds[i].flashPhase;
                analogWrite(leds[i].pin, leds[i].flashPhase ? 255 : LED_DIM_FLOOR);
                leds[i].lastToggleTime = now;
            }
            continue;
        }

        // Smooth fade handling
        if (leds[i].fading) {
            if (now - leds[i].lastFadeUpdate < 10) continue;
            leds[i].lastFadeUpdate = now;

            if (leds[i].fadeMode) {
                // Continuous breathing
                leds[i].fadeCurrent += leds[i].fadeStep;
                if (leds[i].fadeCurrent >= 255) {
                    leds[i].fadeCurrent = 255;
                    leds[i].fadeStep = -leds[i].fadeStep;
                } else if (leds[i].fadeCurrent <= LED_DIM_FLOOR) {
                    leds[i].fadeCurrent = LED_DIM_FLOOR;
                    leds[i].fadeStep = -leds[i].fadeStep;
                }
            } else {
                // One-off fade toward target
                if (leds[i].fadeCurrent < leds[i].fadeTarget) {
                    leds[i].fadeCurrent = min(255, leds[i].fadeCurrent + 5);
                } else if (leds[i].fadeCurrent > leds[i].fadeTarget) {
                    leds[i].fadeCurrent = max(0, leds[i].fadeCurrent - 5);
                }
            }

            analogWrite(leds[i].pin, leds[i].fadeCurrent);
        }
    }
}

```

6.4 Flashing the Firmware

1. **Connect the Pico 2** to your PC via USB while holding the **BOOTSEL** button
2. The Pico 2 will appear as a USB drive named **RPI-RP2**

3. In Arduino IDE, click the **Upload** button (right arrow)
4. The IDE will compile the code and flash it to the Pico 2
5. When complete, the Pico 2 will reboot automatically
6. Open **Device Manager** on Windows → “Human Interface Devices” → you should see a new “USB Input Device” (the joystick)
7. Open **joy.cpl** (Win+R, type joy.cpl) — the Pico 2 should appear as a game controller with 19 buttons

6.5 Verifying the Firmware Works

Before connecting SimHub, test the basic joystick functionality:

1. Open **joy.cpl**
2. Select the Pico 2 device, click **Properties**
3. Press each physical button, toggle switch, and encoder click — the corresponding numbered button should light up in the test dialog
4. Rotate each encoder — you should see brief flashes on the corresponding “up” and “down” buttons

If any input doesn’t register, check your wiring against the pin map and verify your solder joints.

7. SimHub Integration — LED & OLED Control

SimHub is the bridge between your racing game and the button box. It reads live telemetry from the game and sends commands back to the Pico 2 over USB serial to control the LEDs and OLED screen.

7.1 SimHub Serial Output Setup

1. **Open SimHub** and go to the “**Arduino**” or “**Custom Serial Devices**” section
2. **Add a new serial device** pointing to the Pico 2’s COM port
3. **Set the baud rate to 115200** (must match the `Serial.begin(115200)` in the Arduino code)
4. **Set the refresh rate to 60Hz** for the OLED display code

7.2 LED Control — NCALC Formulas

SimHub uses **NCALC** formulas to decide when each LED should turn on, off, or flash. Each formula outputs a text command that the Pico 2 understands. **Set all LED rows to “Changes Only”** in SimHub to prevent spamming the serial port with identical commands.

LED1 — Red: Low Fuel Warning Flashes smoothly when you have 3 laps of fuel or less remaining.

```
NCALC Formula:
IF([ATSRHubMain.Telemetry.Vehicle.FuelLevel] / [DataCorePlugin.Computed.Fuel_LitersPerLap] <= 3,
↵ 'LED1_FADE_FLASH_100', 'LED1_OFF') + '\n'
```

What it does: Divides current fuel by fuel consumption per lap. If the result is ≤ 3 laps, sends LED1_FADE_FLASH_100 (smooth breathing flash, 100ms half-cycle). Otherwise sends LED1_OFF.

LED2 — Green: Tyre Wear Warning Flashes smoothly when tyre wear drops below 40% remaining.

NCALC Formula:

```
IF([DataCorePlugin.GameData.TyresWearMin] < 40, 'LED2_FADE_FLASH_100', 'LED2_OFF') + '\n'
```

What it does: Reads the minimum tyre wear across all four tyres. If below 40%, sends LED2_FADE_FLASH_100. Otherwise off.

LED3 — Blue: Blue Flag Warning Flashes smoothly when a blue flag is shown.

NCALC Formula:

```
IF([DataCorePlugin.GameData.Flag_Blue], 'LED3_FADE_FLASH_100', 'LED3_OFF') + '\n'
```

What it does: Checks if the blue flag boolean is true. If so, flashes LED3.

LED4 — Yellow: Yellow Flag Warning Flashes smoothly when a yellow flag is shown.

NCALC Formula:

```
IF([DataCorePlugin.GameData.Flag_Yellow], 'LED4_FADE_FLASH_100', 'LED4_OFF') + '\n'
```

What it does: Checks if the yellow flag boolean is true. If so, flashes LED4.

Best Lap Celebration — All 4 LEDs Chase Sequence When you set a new best lap, all 4 LEDs light up in a chasing sequence with staggered timing.

Create 4 separate NCALC rows (one per LED):

```
LED1: IF(changed(2000, [DataCorePlugin.GameData.BestLapTime]), 'LED1_ON', 'LED1_OFF') + '\n'
LED2: IF(changed(1700, [DataCorePlugin.GameData.BestLapTime]), 'LED2_ON', 'LED2_OFF') + '\n'
LED3: IF(changed(1400, [DataCorePlugin.GameData.BestLapTime]), 'LED3_ON', 'LED3_OFF') + '\n'
LED4: IF(changed(1100, [DataCorePlugin.GameData.BestLapTime]), 'LED4_ON', 'LED4_OFF') + '\n'
```

What it does: The changed() function detects when the best lap time value updates. The different millisecond values (2000, 1700, 1400, 1100) create a staggered delay so the LEDs light up in sequence: red → green → blue → yellow. Each LED stays on for the duration of the changed() window, creating a chase effect.

7.3 OLED Display — JavaScript Code

The OLED display is driven by a JavaScript snippet in SimHub that runs at 60Hz. It assembles a DISP|Line1|Line2|Line3|Line4\n string and sends it to the Pico 2 over serial.

Setup in SimHub

1. Go to **Arduino → Custom Serial Devices** (or the appropriate section for your setup)
2. Create a new JavaScript action
3. Set **Run Once** code and **Loop** code separately
4. Set refresh rate to **60Hz**

Run Once Code (Initialises Variables) Paste this into the “Run Once” section:

```
var lastBestLap = null;
var bestLapTimer = 0;
```

```

var lastPitDur = null;
var pitTimer = 0;
var lastTyreFL = null;
var flTimer = 0;
var lastTyreFR = null;
var frTimer = 0;
var lastTyreRL = null;
var rlTimer = 0;
var lastTyreRR = null;
var rrTimer = 0;
var cycleStart = 0;
var pitsCompleted = 0;
var lastCurrentLap = 0;

```

Loop Code (Runs Every Frame) Paste this into the “Loop” or main JavaScript section:

```

/*
=====
SIM RACING BUTTON BOX - DISPLAY CONTROLLER
=====

Game: Automobilista 2 (AMS2) via SimHub
Hardware: Raspberry Pi Pico + 4-line OLED (128x64)

OVERVIEW:
This script drives a 4-line OLED display on a custom button box.
It auto-detects race type (laps vs timed) and class structure
(single vs multiclass) to show relevant data without manual switching.

LINE 1 — Position info:
- Single-class:  P[Position] L[Lap]
- Multiclass:    P[Position] C[ClassPos] L[Lap]

LINE 2 — Fuel info (rotates every 10 seconds):
- Slot 0 (0-10s): F[Fuel] E[Litres to add at next pit stop]
- Slot 1 (10-20s): F[Fuel] T[Total litres needed to finish race]

LINE 3 — Pit window:
- Pit [laps remaining before fuel runs out]

LINE 4 — Status / overrides (rotates every 10 seconds):
- Priority overrides: live pit timer, frozen pit exit time,
  BEST LAP (4s flash), tyre wear warnings (15s flash).
- Slot 0: Lap counter L[current/total] OR time remaining T[mm:ss]
- Slot 1: Pit stop progress PIT[completed/total]

NOTE: LED rows are controlled separately in SimHub NCALC formulas.
      Set all LED rows to "Changes only" to prevent animation spam.
=====
*/

// --- Helper: Convert SimHub TimeSpan objects or strings to plain seconds ---

```

```

function toSeconds(v) {
    if (typeof v === 'number') return v;
    if (v && typeof v.TotalSeconds === 'number') return v.TotalSeconds;
    if (typeof v === 'string' && v.indexOf(':') > -1) {
        var p = v.split(':');
        return parseInt(p[0]) * 3600 + parseInt(p[1]) * 60 + parseFloat(p[2]);
    }
    return 0;
}

// --- Helper: Format seconds as mm:ss.000 for pit timers ---
function fmtPit(sec) {
    var m = Math.floor(sec / 60);
    var s = Math.floor(sec % 60);
    var ms = Math.floor((sec - Math.floor(sec)) * 1000);
    return format(m, '00') + ':' + format(s, '00') + '.' + format(ms, '000');
}

// --- Timing anchors ---
var now = Date.now();
if (cycleStart === 0) cycleStart = now;
var elapsed = now - cycleStart;

// --- Pull all SimHub properties ---
var remainingLaps = $prop('DataCorePlugin.GameData.RemainingLaps');
var fuelPerLap = $prop('DataCorePlugin.Computed.Fuel_LitersPerLap');
var fuel = $prop('DataCorePlugin.GameData.Fuel');
var position = $prop('DataCorePlugin.GameData.Position');
var currentLap = $prop('DataCorePlugin.GameData.CurrentLap');
var isInPit = $prop('DataCorePlugin.GameData.IsInPit');
var isInPitSince = $prop('DataCorePlugin.GameData.IsInPitSince');
var lastPitStopDuration = $prop('DataCorePlugin.GameData.LastPitStopDuration');
var bestLapTime = $prop('DataCorePlugin.GameData.BestLapTime');
var sessionTime = $prop('DataCorePlugin.GameData.SessionTimeLeft');
var lastLapTime = $prop('DataCorePlugin.GameData.LastLapTime');
var tyreFL = $prop('DataCorePlugin.GameData.TyreWearFrontLeft');
var tyreFR = $prop('DataCorePlugin.GameData.TyreWearFrontRight');
var tyreRL = $prop('DataCorePlugin.GameData.TyreWearRearLeft');
var tyreRR = $prop('DataCorePlugin.GameData.TyreWearRearRight');
var tankCapacity = $prop('DataCorePlugin.GameData.CarSettings_MaxFUEL');

// --- Convert TimeSpan properties to usable numbers ---
var stl = toSeconds(sessionTime);
var llt = toSeconds(lastLapTime);
var blt = toSeconds(bestLapTime);
var lpd = toSeconds(lastPitStopDuration);

// --- Session reset: if lap counter went backwards, new race started ---
if (lastCurrentLap > 0 && currentLap < lastCurrentLap) {
    pitsCompleted = 0;
}

```

```

lastCurrentLap = currentLap;

// =====
// LINE 1 — Position / Class Position / Lap
// Detects multiclass by scanning leaderboard CarClass strings.
// =====
var playerClass = $prop('DataCorePlugin.GameData.CarClass');
var isMulticlass = false;

if (playerClass && playerClass !== '') {
    for (var i = 1; i <= 30; i++) {
        var idx = i < 10 ? '0' + i : '' + i;
        var checkClass = $prop('GarySwallowDataPlugin.Leaderboard.Position' + idx + '.CarClass');
        if (checkClass && checkClass !== '' && checkClass !== playerClass) {
            isMulticlass = true;
            break;
        }
    }
}

var line1 = '';
if (isMulticlass) {
    var classPos = 1;
    var maxCheck = Math.min(position, 70);
    for (var i = 1; i < maxCheck; i++) {
        var idx = i < 10 ? '0' + i : '' + i;
        var checkClass = $prop('GarySwallowDataPlugin.Leaderboard.Position' + idx + '.CarClass');
        if (checkClass === null || checkClass === undefined || checkClass === '') continue;
        if (checkClass === playerClass) classPos++;
    }
    line1 = 'P' + position + ' C' + classPos + ' L' + currentLap;
} else {
    line1 = 'P' + position + ' L' + currentLap;
}

// =====
// TOTAL FUEL NEEDED TO FINISH
// Lap race: RemainingLaps × FuelPerLap
// Time race: (SessionTimeLeft ÷ LastLapTime) × FuelPerLap
// =====
var totalFuelNeeded = 0;
if (remainingLaps > 0 && fuelPerLap > 0 && isFinite(fuelPerLap) && isFinite(remainingLaps)) {
    totalFuelNeeded = Math.ceil(remainingLaps * fuelPerLap);
} else {
    var lapTime = llt > 0 ? llt : blt;
    if (lapTime > 0 && stl > 0 && fuelPerLap > 0 && isFinite(fuelPerLap) && isFinite(stl) &&
        isFinite(lapTime)) {
        totalFuelNeeded = Math.ceil((stl / lapTime) * fuelPerLap);
    }
}

```

```

// =====
// LINE 2 — Fuel to add at next pit stop (capped by tank size)
// =====
var fuelToAdd = 0;
if (totalFuelNeeded > fuel && isFinite(totalFuelNeeded) && isFinite(fuel)) {
    if (tankCapacity > 0 && isFinite(tankCapacity)) {
        fuelToAdd = Math.min(tankCapacity, totalFuelNeeded - fuel);
    } else {
        fuelToAdd = totalFuelNeeded - fuel;
    }
    fuelToAdd = Math.max(0, Math.ceil(fuelToAdd));
}

// =====
// LINE 2 ROTATION (10-second cycle)
// Slot 0 → E = Litres to add at next pit stop
// Slot 1 → T = Total litres needed to finish race
// =====
var fuelSlot = Math.floor(elapsed / 10000) % 2;
var fuelLabel = 'E';
var fuelDisplay = fuelToAdd;

if (fuelSlot === 1 && totalFuelNeeded > 0 && isFinite(totalFuelNeeded)) {
    fuelLabel = 'T';
    fuelDisplay = totalFuelNeeded;
}

// =====
// LINE 4 CHANGE DETECTION (temporary overrides)
// These timers create timed flashes when values change:
// - Best Lap: 4 seconds
// - Pit stop: 4 seconds (increments pitsCompleted counter)
// - Tyre wear: 15 seconds per corner (only if wear < 40%)
// =====
if (bestLapTime !== lastBestLap && blt > 0) {
    bestLapTimer = now + 4000;
}
lastBestLap = bestLapTime;

if (lastPitStopDuration !== lastPitDur && lpd > 0) {
    pitTimer = now + 4000;
    pitsCompleted++;
}
lastPitDur = lastPitStopDuration;

if (tyreFL !== lastTyreFL && tyreFL < 40) { flTimer = now + 15000; }
lastTyreFL = tyreFL;

if (tyreFR !== lastTyreFR && tyreFR < 40) { frTimer = now + 15000; }
lastTyreFR = tyreFR;

```

```

if (tyreRL !== lastTyreRL && tyreRL < 40) { rlTimer = now + 15000; }
lastTyreRL = tyreRL;

if (tyreRR !== lastTyreRR && tyreRR < 40) { rrTimer = now + 15000; }
lastTyreRR = tyreRR;

// =====
// LINE 4 BUILDER
// Priority order:
// 1. Live pit timer
// 2. Frozen last pit duration
// 3. BEST LAP flash
// 4. Tyre wear warnings (FL, FR, RL, RR)
// 5. 10-second cycle: Lap counter OR Pit stop progress
// =====
var line4 = '';
if (isInPit) {
    line4 = fmtPit(isInPitSince);
} else if (now < pitTimer) {
    line4 = fmtPit(lpd);
} else if (now < bestLapTimer) {
    line4 = 'BEST LAP';
} else if (now < flTimer) {
    line4 = 'FRONT LEFT';
} else if (now < frTimer) {
    line4 = 'FRONT RIGHT';
} else if (now < rlTimer) {
    line4 = 'REAR LEFT';
} else if (now < rrTimer) {
    line4 = 'REAR RIGHT';
} else {
    var slot = Math.floor(elapsed / 10000) % 2;

    if (slot === 0) {
        // Lap or time info
        if (remainingLaps > 0) {
            var totallaps = currentLap + remainingLaps - 1;
            line4 = 'L' + currentLap + '/' + totallaps;
        } else if (stl > 0) {
            if (stl >= 3600) {
                var hrs = Math.floor(stl / 3600);
                var mins = Math.floor((stl % 3600) / 60);
                var secs = Math.floor(stl % 60);
                line4 = 'T' + hrs + ':' + format(mins, '00') + ':' + format(secs, '00');
            } else {
                var mins = Math.floor(stl / 60);
                var secs = Math.floor(stl % 60);
                line4 = 'T' + mins + ':' + format(secs, '00');
            }
        }
    }
} else {

```

```

// Stops needed: PIT X/X format
if (totalFuelNeeded > 0 && tankCapacity > 0 && isFinite(tankCapacity) &&
    isFinite(totalFuelNeeded) && isFinite(fuel)) {
    var remainingStops = Math.max(0, Math.ceil((totalFuelNeeded - fuel) / tankCapacity));
    var totalStops = pitsCompleted + remainingStops;
    line4 = 'PIT ' + pitsCompleted + '/' + totalStops;
} else {
    // Fallback to lap/time if no fuel data yet
    if (remainingLaps > 0) {
        var totallaps = currentLap + remainingLaps - 1;
        line4 = 'L' + currentLap + '/' + totallaps;
    } else if (stl > 0) {
        if (stl >= 3600) {
            var hrs = Math.floor(stl / 3600);
            var mins = Math.floor((stl % 3600) / 60);
            var secs = Math.floor(stl % 60);
            line4 = 'T' + hrs + ':' + format(mins, '00') + ':' + format(secs, '00');
        } else {
            var mins = Math.floor(stl / 60);
            var secs = Math.floor(stl % 60);
            line4 = 'T' + mins + ':' + format(secs, '00');
        }
    }
}

// =====
// LINE 3 — Pit laps remaining before fuel runs out
// =====
var pitLaps = 0;
if (fuelPerLap > 0 && isFinite(fuelPerLap) && isFinite(fuel)) {
    pitLaps = Math.round(fuel / fuelPerLap);
}

// =====
// ASSEMBLE SERIAL STRING AND SEND TO PICO
// Format: DISP|Line1|Line2|Line3|Line4\n
// =====
return 'DISP|' + line1 +
    '|F|' + format(fuel, '0.0') + ' ' + fuelLabel + fuelDisplay +
    '|Pit ' + pitLaps +
    '|' + line4 + '\n';

```

7.4 What the Display Shows

Line	Content	Example
Line 1	Position, Class Position (multiclass), Lap	P3 C1 L12 or P3 L12

Table 13 – continued

Line	Content	Example
Line 2	Fuel amount + fuel to add OR total needed	F12.5 E8 or F12.5 T45
Line 3	Laps until pit needed	Pit 4
Line 4	Status: pit timer, best lap, tyre wear, laps/time, pit stops	BEST LAP, FRONT LEFT, L12/25, PIT 1/2

Line 2 and Line 4 rotate every 10 seconds between two information slots, so you see both fuel-to-add and total-fuel-needed, and both lap-counter and pit-stop-progress, without cluttering the screen.

Priority overrides on Line 4: If you're in the pits, it shows the live pit timer. If you just finished a pit stop, it shows the duration for 4 seconds. If you set a best lap, it shows BEST LAP for 4 seconds. If tyre wear drops below 40% on any corner, it shows that corner's name for 15 seconds. Only if none of these are active does it show the rotating lap/time info.

7.5 SimHub Settings Summary

Setting	Value
Serial baud rate	115200
OLED refresh rate	60Hz
LED NCALC rows	Set to Changes Only
LED commands	LED1_FADE_FLASH_100, LED1_OFF, LED1_ON, etc.
OLED command prefix	DISP\

8. Button Virtualization Setup

This section is essential if your sim racing wheel already occupies joystick buttons 1–32. Without virtualization, your button box buttons (also numbered 1–19) will conflict with your wheel. The solution is to shift all button box inputs to higher button numbers (33–51) using vJoy, Joystick Gremlin, and HIDHide.

8.1 What This Setup Does

The Problem:

- Your sim racing wheel occupies joystick buttons 1–32.
- Your Pico 2 button box also outputs buttons 1–19.
- When both are connected, games see overlapping button numbers, causing conflicts.
- Joystick GremlinEx (the fork) cannot read the Pico 2's HID descriptor properly (blank left panel).

The Solution:

- Use **original Joystick Gremlin** (not Ex) — it correctly enumerates the Arduino Joystick Library device.

- Map all Pico 2 buttons to **vJoy Device 1, Buttons 33–51**.
- Use **HIDHide** to hide the physical Pico 2 from games.
- Games only see the **vJoy Device**, which contains your wheel's buttons (1–32) and your button box's shifted buttons (33+).
- Use an **AutoHotkey wrapper** to start Gremlin hidden in the system tray on boot.
- Use **Gremlin's built-in Automation** to auto-activate the profile when your sim racing game launches.

8.2 Software Installation

Software	Version	Download Source
vJoy	2.1.9.1	SourceForge (official)
Joystick Gremlin	R15 (original)	github.com/WhiteMagic/JoystickGremlin/releases
HIDHide	Latest	github.com/ViGEm/HidHide/releases
AutoHotkey	v1.1	autohotkey.com

Install vJoy

1. Download **vJoy 2.1.9.1** from SourceForge.
2. Run the installer as **Administrator**.
3. **Reboot** when prompted.

Configure vJoy Device 1

1. Open **Configure vJoy** from the Start Menu.
2. Enable **Device 1**.
3. Set the following:
 - **Axes:** 0 (or 1 if a game complains)
 - **Buttons:** 128
 - **POVs:** 0
4. Click **Apply**.
5. **Reboot** again.

Verify vJoy

1. Press Win + R, type joy.cpl, hit Enter.
2. You should see **vJoy Device** in the list.
3. Click **Properties** — the test dialog only shows buttons 1–32 in the UI, but all 128 exist behind the scenes.

Note: joy.cpl is an ancient Windows dialog. It cannot visually display buttons 33+, but they are fully functional for games and DirectInput tools.

8.3 Joystick Gremlin (Original) Setup

Critical: Use the **original** Joystick Gremlin, **not** Joystick GremlinEx. GremlinEx has a known compatibility bug with Arduino Joystick Library devices and will show a blank left panel.

Install & Launch

1. Download **Joystick Gremlin R15** (or latest stable) from the official GitHub releases.
2. Extract the ZIP to a permanent folder, e.g.:

```
C:\Tools\JoystickGremlin
```

3. Run JoystickGremlin.exe as **Administrator**.

Create the Button Box Profile

1. Plug in your **Pico 2 button box**.
2. In Gremlin, click the “**Pico 2**” tab at the top.
3. The left panel should show **Axes** and **Buttons**.
4. Click “**Button 1**” in the left panel (it highlights in blue).
5. Click “**New Action Sequence**” at the bottom-right.
6. In the right panel, click “**Add Action**”.
7. Select “**Map to vJoy**” from the dropdown.
8. Configure:
 - **vJoy Device:** 1
 - **vJoy Button:** 33
9. Click **OK**.
10. Repeat for every button on your box:
 - Button 2 → vJoy Button 34
 - Button 3 → vJoy Button 35
 - ...and so on, up to Button 19 → vJoy Button 51.

Mapping Reference:

Pico 2 Button	Function	vJoy Button
1	Encoder 1 Up	33
2	Encoder 1 Down	34
3	Encoder 1 Click	35
4	Encoder 2 Up	36
5	Encoder 2 Down	37
6	Encoder 2 Click	38
7	Encoder 3 Up	39
8	Encoder 3 Down	40
9	Encoder 3 Click	41
10	Toggle 1 Up	42
11	Toggle 1 Down	43
12	Toggle 2 Up	44
13	Toggle 2 Down	45
14	Toggle 3 Up	46

Table 16 – continued

Pico 2 Button	Function	vJoy Button
15	Toggle 3 Down	47
16	Button 1	48
17	Button 2	49
18	Button 3	50
19	Button 4	51

Save the Profile

1. Go to **File** → **Save As...**
2. Name it something descriptive, e.g.:

Button Box to vJoy Device 1 (33-51).xml

3. Save it inside Gremlin’s Profiles folder or wherever you prefer.

Test the Mapping

1. Click the **Activate** button (joystick with green play icon) in the toolbar.
2. Bottom-left status should change to **“Running”**.
3. Open **Input Viewer** (graph icon in toolbar).
4. Select **vJoy Device 1** from the dropdown.
5. Press buttons on your button box.
6. You should see bars light up at buttons **33, 34, 35**, etc.

8.4 HIDHide Configuration

HIDHide prevents games from seeing your physical Pico 2 directly. Without this, games would see **both** the physical Pico 2 (buttons 1–19) and the vJoy device (buttons 33+), causing double-inputs.

Install HIDHide

1. Download the latest release from github.com/ViGEm/HidHide/releases.
2. Install and **reboot**.

Configure HIDHide

1. Open **HIDHide Configuration Client** (requires Administrator).
2. Check **“Enable device hiding”** at the top.
3. Go to the **Applications** tab:
 - Click the **+** button.
 - Browse to and add your JoystickGremlin.exe.
 - This is **critical** — without it, Gremlin cannot see the Pico 2 either.
4. Go to the **Devices** tab:
 - Find **“Pico 2”** in the list.
 - Check the box next to it.
 - **Do NOT** check your wheel unless you also want to virtualize it.
5. Click **Apply**.

Verify HIDHide

1. Open joy.cpl (Win + R → joy.cpl).
2. The **Pico 2 should be gone** from the list.
3. **vJoy Device** should still be visible.
4. If Pico 2 still shows, close joy.cpl and reopen it.

8.5 AutoHotkey Tray Wrapper

Original Joystick Gremlin has **no native** “minimize to system tray” or “start minimized” feature. This wrapper script solves that.

Install AutoHotkey

1. Download **AutoHotkey v1.1** from autohotkey.com.
2. Install it.

Create the Script

1. Open **Notepad**.
2. Paste the following (adjust the gremlinPath to match your install):

```
#Persistent
#NoEnv
DetectHiddenWindows, On

; === CONFIG ===
gremlinPath := "C:\Tools\JoystickGremlin\JoystickGremlin.exe"
; =====

; Derive Gremlin's folder so it finds its _internal DLLs
SplitPath, gremlinPath,, gremlinDir
SetWorkingDir %gremlinDir%

; Tray menu
Menu, Tray, Icon, shell32.dll, 44
Menu, Tray, Tip, Joystick Gremlin (Starting...)
Menu, Tray, Add, Show Gremlin, ShowGremlin
Menu, Tray, Add, Hide Gremlin, HideGremlin
Menu, Tray, Add, Exit, ExitScript

hidden := false
gremlinPID := 0

; Check if Gremlin is already running from a failed boot attempt
Process, Exist, JoystickGremlin.exe
if (ErrorLevel != 0) {
    gremlinPID := ErrorLevel
    SetTimer, TryHideGremlin, 3000
} else {
    ; Launch minimized (NOT hidden -- Hide crashes Gremlin's DLL init)
    Run, "%gremlinPath%", %gremlinDir%, Min, gremlinPID
```

```

    SetTimer, TryHideGremlin, 5000
}

return

TryHideGremlin:
    if (gremlinPID = "" or gremlinPID = 0) {
        return
    }

    ; If already hidden, stop the timer permanently
    if (hidden) {
        SetTimer, TryHideGremlin, Off
        return
    }

    ; Check if window exists yet
    if (!WinExist("ahk_pid " . gremlinPID)) {
        return
    }

    ; Restore first (WinHide can fail on minimized windows during boot)
    WinRestore, ahk_pid %gremlinPID%
    Sleep, 500
    WinHide, ahk_pid %gremlinPID%
    Sleep, 500

    ; Verify the window is ACTUALLY hidden before trusting it
    WinGet, Style, Style, ahk_pid %gremlinPID%
    if (Style & 0x10000000) {
        ; Window is still visible -- hide failed, keep trying
        return
    }

    ; Confirmed hidden
    hidden := true
    SetTimer, TryHideGremlin, Off
    Menu, Tray, Tip, Joystick Gremlin (Running)
return

ShowGremlin:
    if (gremlinPID) {
        WinShow, ahk_pid %gremlinPID%
        WinRestore, ahk_pid %gremlinPID%
        WinActivate, ahk_pid %gremlinPID%
        hidden := false
        Menu, Tray, Tip, Joystick Gremlin (Visible)
    }
return

HideGremlin:

```

```

    if (gremlinPID) {
        WinRestore, ahk_pid %gremlinPID%
        Sleep, 200
        WinHide, ahk_pid %gremlinPID%
        hidden := true
        Menu, Tray, Tip, Joystick Gremlin (Running)
    }
return

ExitScript:
    ; Kill Gremlin before exiting the wrapper
    if (gremlinPID) {
        Process, Close, %gremlinPID%
    }
    Process, Close, JoystickGremlin.exe
    ExitApp

```

3. Save the file as GremlinTray.ahk in the **same folder** as JoystickGremlin.exe.

Important: Save this file in the same folder as JoystickGremlin.exe. This ensures any AutoHotkey log files are generated there instead of your startup folder.

Compile to .exe

1. Right-click GremlinTray.ahk in File Explorer.
2. Click **Compile Script**.
3. This creates GremlinTray.exe in the same folder.

Set to Start on Boot

1. Press Win + R, type shell:startup, hit Enter.
2. **Create a shortcut** to GremlinTray.exe and put the shortcut in this folder.
3. **Do NOT** put the .exe itself in the startup folder — only a shortcut.
4. **Remove any old Joystick Gremlin shortcuts** from this folder — the wrapper replaces them.

How the Wrapper Works

- **Boot** → GremlinTray.exe starts automatically.
- **Gremlin launches** → window appears briefly for ~2 seconds.
- **Wrapper hides it** → window vanishes from taskbar.
- **Tray icon remains** → right-click the star icon to show/hide/exit.
- Gremlin runs silently in the background using ~0% CPU.
- **Exit from tray** → wrapper kills Gremlin process before closing.

Note on cold boots: If Windows displays a “Getting ready” screen on startup, the desktop window manager may not be ready when the script first tries to hide Gremlin. The script detects this and retries every 5 seconds until the hide succeeds, up to a maximum of one minute.

8.6 Gremlin Automation (Auto-Activate)

This removes the final manual step — clicking the **Activate** button every time you launch a game.

Configure Automation

1. In Joystick Gremlin, go to **Tools** → **Options** → **Automation**.
2. Toggle “**Enable auto loading**” to **On**.
3. For each sim racing game you play:
 - Click “**New Entry**”.
 - Click “**Select Profile**” → browse to your saved profile.
 - Click “**Browse Executable**” → browse to the game’s .exe file.
 - Toggle the entry to **On**.
4. Toggle “**Remain active on focus loss**” to **On**.
 - This keeps the profile active when you alt-tab out of the game.

Example Entries

Game	Executable Path	Profile
Automobilista 2	D:/SteamLibrary/steamapps/common/Automobilista 2/AMS2.exe	Button Box to vJoy Device 1 (33-51).xml
Automobilista 2 (AVX)	D:/SteamLibrary/steamapps/common/Automobilista 2/AMS2AVX.exe	Button Box to vJoy Device 1 (33-51).xml

Why two entries for AMS2? The game can launch via either executable depending on settings. Mapping both ensures the profile always loads.

8.7 How It All Works Together

Boot Sequence

```
Windows starts
|-- GremlinTray shortcut launches (from shell:startup)
|-- Joystick Gremlin starts, hides to tray
    |-- Gremlin is running but NOT activated yet
        |-- HIDHide is already filtering the Pico 2
            |-- vJoy Device 1 exists with 128 buttons
```

Game Launch Sequence

```
You launch Automobilista 2
|-- Gremlin detects AMS2.exe / AMS2AVX.exe
    |-- Gremlin auto-loads your profile
        |-- Gremlin auto-ACTIVATES the profile
            |-- Pico 2 buttons are now remapped to vJoy 33+
                |-- Game only sees vJoy Device (Pico 2 is hidden)
                    |-- You bind controls using vJoy buttons 33+
```

Shutdown Sequence

```
You close the game
|-- Gremlin detects the executable is gone
    |-- Gremlin deactivates the profile
        |-- Inputs stop being remapped
```

```
|-- System returns to idle state
```

9. Final Assembly & Testing

This section covers the final physical assembly of the enclosure and a complete testing procedure to verify every function before using the box in a race.

9.1 Final Mechanical Assembly

1. **Route all wires** from the faceplate components down into the enclosure toward the Pico 2 breakout board location.
2. **Mount the Pico 2 breakout board** on standoffs in the centre-bottom of the enclosure base.
3. **Connect the USB cable** through the hole in the back panel. Use a rubber grommet to prevent the cable from chafing against the MDF edge.
4. **Neaten the wiring:** Use zip ties or cable clips to bundle wires and keep them away from the faceplate mounting screw holes.
5. **Attach the faceplate** to the MDF frame using M3 or M4 screws into the threaded inserts.
6. **Final visual check:**
 - No bare wires touching the metal faceplate
 - No loose screws inside the enclosure
 - USB cable has strain relief (not pulling on the Pico 2 connector)
 - All components are firmly mounted and don't rattle

9.2 Power-On Test Sequence

Perform these tests in order. Do not skip steps.

Test 1: Verify USB Connection

1. Plug the USB cable into your PC.
2. The Pico 2 should appear in Device Manager under “Human Interface Devices” as a USB Input Device.
3. Open joy.cpl (Win+R -> joy.cpl).
4. You should see the Pico 2 listed as a game controller.

Test 2: Verify Joystick Inputs (Without Gremlin)

1. In joy.cpl, select the Pico 2, click **Properties**.
2. Test every input:
 - **Toggle switches:** Flip each direction — two buttons should light up per switch (top and bottom).
 - **LED buttons:** Press each button — one button should light up per press.
 - **Encoder clicks:** Push each encoder shaft in — one button should light up per encoder.
 - **Encoder rotation:** Rotate each encoder clockwise and counter-clockwise — you should see brief flashes on the corresponding “up” and “down” buttons.
3. If any input doesn't register, check your wiring against the pin map and verify solder joints.

Test 3: Verify vJoy Exists

- Win + R -> joy.cpl -> **vJoy Device** should be listed.

Test 4: Verify HIDHide Works

- joy.cpl -> **Pico 2 should NOT be listed** (it should be hidden).
- If Pico 2 still appears, check HIDHide's Devices tab and click Apply again.

Test 5: Verify Gremlin Mapping

1. Launch Gremlin manually (or Show Gremlin from tray).
2. Activate the profile manually.
3. Open Gremlin's **Input Viewer**.
4. Select **vJoy Device 1**.
5. Press button box buttons -> should see activity at 33+.

Test 6: Verify SimHub LED Commands

1. Open SimHub and start a game session (or use SimHub's test mode).
2. Trigger each LED condition:
 - **Low fuel:** Drive until fuel drops to ≤ 3 laps — LED1 (red) should fade-flash.
 - **Tyre wear:** Drive until any tyre drops below 40% — LED2 (green) should fade-flash.
 - **Blue flag:** Get lapped or approach a faster car — LED3 (blue) should fade-flash.
 - **Yellow flag:** Trigger a yellow flag situation — LED4 (yellow) should fade-flash.
 - **Best lap:** Set a new best lap — all 4 LEDs should chase in sequence.
3. If an LED doesn't respond, check:
 - The NCALC formula is set to **Changes Only**
 - The serial device in SimHub is pointing to the correct COM port at 115200 baud
 - The Arduino code was flashed correctly

Test 7: Verify OLED Display

1. With SimHub running and connected to a game, the OLED should show live data.
2. Check that Line 1 shows position and lap.
3. Check that Line 2 shows fuel data (rotates every 10 seconds).
4. Check that Line 3 shows pit laps remaining.
5. Check that Line 4 shows status info (or overrides when triggered).
6. If the screen is blank:
 - Check the I2C wires (SDA on GP16, SCL on GP17)
 - Verify the OLED address is 0x3C in both the Arduino code and the physical module
 - Check that SimHub is sending **DISP|** commands

Test 8: Full End-to-End

1. **Reboot** your PC.
2. Verify the star icon appears in the system tray.
3. Launch your sim racing game.
4. Go to controls/bindings.
5. Press every button on your button box.
6. Each should register as a **unique, non-conflicting** input (buttons 33-51).

10. Troubleshooting & Reference

10.1 Wiring & Hardware Problems

Problem	Likely Cause	Fix
Button presses register randomly / ghost inputs	LED and switch grounds bridged at the button	Cut the bridge wire. Each button needs 4 separate wires. See Section 5.1.
LED doesn't light at all	Resistor not connected, LED wired backwards, or wrong pin	Check resistor is in series with LED+. Check LED polarity. Verify pin map.
LED is very dim	Resistor value too high	Use 220-470 Ohm. 330 Ohm is ideal.
Encoder doesn't register rotation	CLK/DT wires swapped or not soldered well	Check wiring against pin map. Resolder if joint looks dull/cracked.
Encoder click doesn't work	SW pin not connected	Check SW wire is seated in the correct terminal.
Toggle switch only works one way	Centre pin not grounded, or one direction wire loose	Verify centre pin goes to GND. Check both top and bottom wires.
OLED screen is completely blank	Wrong I2C address, or SDA/SCL swapped	Verify address 0x3C. Check SDA=GP16, SCL=GP17.
OLED shows garbage / corrupted text	Loose I2C connection or insufficient power	Resolder SDA/SCL. Check 3V3 terminal is tight.
Pico 2 not detected by Windows	Bad USB cable (power-only, no data), or dead Pico	Try a different USB cable (must have data lines). Try a different USB port.

10.2 Software Problems

Problem	Likely Cause	Fix
Gremlin shows “Not Running” when I launch the game	Gremlin isn’ t running, or wrong executable path, or permission mismatch	Check tray icon. Re-browse to exact .exe in Automation tab. Run Gremlin as Administrator.
Game doesn’ t see vJoy buttons	Profile not activated, or HIDHide hiding vJoy too, or vJoy service stopped	Check Gremlin status = “Running”. In HIDHide, uncheck vJoy Device. In services.msc, start vJoy.
Gremlin can’ t see Pico 2 (blank left panel)	HIDHide blocking Gremlin, or using GremlinEx	Whitelist Joy-stickGremlin.exe in HIDHide Applications. Use original Joystick Gremlin, not Ex.
Buttons still conflict with wheel	Offset too low, or game reading physical Pico 2 directly	Start button box at 33 or higher. Verify HIDHide is enabled and Pico 2 is checked in hidden devices.
Tray wrapper doesn’ t start on boot	Wrong startup folder, Windows blocked it, or Gremlin window stays visible	Put only a shortcut in shell:startup . Check Windows Security for blocks. Click Hide Gremlin from tray.
SimHub LEDs don’ t respond	Wrong COM port, wrong baud rate, or NCALC not set to Changes Only	Verify COM port and 115200 baud. Set all LED rows to Changes Only.
OLED doesn’ t update	SimHub not sending DISP commands, or JavaScript error	Check SimHub serial device settings. Test the JavaScript in SimHub’ s script editor.

10.3 How to Revert / Uninstall

Temporarily Disable

1. **Exit GremlinTray** from the system tray (right-click -> Exit).
2. Open **HIDHide Configuration Client**.
3. Uncheck “**Enable device hiding**”.
4. Your Pico 2 will be visible to games again as a normal joystick.

Fully Uninstall

1. **Remove from startup:** Win + R -> shell:startup -> delete the GremlinTray.exe shortcut.
2. **Uninstall HIDHide:** Windows Settings -> Apps -> HIDHide -> Uninstall -> Reboot.
3. **Uninstall vJoy:** Windows Settings -> Apps -> vJoy -> Uninstall -> Reboot.
4. **Delete Joystick Gremlin:** Delete the C:\Tools\JoystickGremlin folder (it's portable).
5. **Delete AutoHotkey wrapper:** Delete GremlinTray.ahk and GremlinTray.exe.
6. **Uninstall AutoHotkey** (optional): Windows Settings -> Apps -> AutoHotkey -> Uninstall.

No button box firmware changes are required for this setup. Your Pico 2 remains exactly as it was before virtualization.

10.4 File Locations Reference

File / Folder	Location	Purpose
Joystick Gremlin	C:\Tools\JoystickGremlin	Main application folder
Gremlin Profiles	C:\Tools\JoystickGremlin\Profiles	Your saved .xml profiles
GremlinTray.ahk	C:\Tools\JoystickGremlin\GremlinTray.ahk	AutoHotkey source script
GremlinTray.exe	C:\Tools\JoystickGremlin\GremlinTray.exe	Compiled tray wrapper
Startup Folder	%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup	Boot launch shortcut
HIDHide	C:\Program Files\Nefarius\HidHide	Default install location
vJoy	C:\Program Files\vJoy	Default install location
AutoHotkey	C:\Program Files\AutoHotkey	Default install location

10.5 Quick Reference Card

What you want to do	How to do it
Check if it's working	Launch game -> Gremlin should auto-activate -> bind controls
Show Gremlin window	Right-click tray star icon -> Show Gremlin
Hide Gremlin window	Right-click tray star icon -> Hide Gremlin

Table 21 – continued

What you want to do	How to do it
Exit everything	Right-click tray star icon -> Exit
Temporarily disable	Open HIDHide -> uncheck “Enable device hiding”
Add a new game to auto-activate	Gremlin -> Tools -> Options -> Automation -> New Entry
Change button mappings	Show Gremlin -> Deactivate -> edit -> Save -> Reactivate
Reboot and test	Reboot -> wait for tray icon -> launch game -> verify

10.6 Button Box Button Reference

Joystick Button	Physical Input	Default Suggested Mapping
1	Encoder 1 Clockwise	Traction Control +
2	Encoder 1 Counter-Clockwise	Traction Control -
3	Encoder 1 Push	TC Reset / Off
4	Encoder 2 Clockwise	ABS +
5	Encoder 2 Counter-Clockwise	ABS -
6	Encoder 2 Push	ABS Reset
7	Encoder 3 Clockwise	Brake Bias +
8	Encoder 3 Counter-Clockwise	Brake Bias -
9	Encoder 3 Push	Brake Bias Reset
10	Toggle 1 Up	Engine Map +
11	Toggle 1 Down	Engine Map -
12	Toggle 2 Up	Anti-Roll Front +
13	Toggle 2 Down	Anti-Roll Front -
14	Toggle 3 Up	Anti-Roll Rear +
15	Toggle 3 Down	Anti-Roll Rear -
16	LED Button 1	Pit Limiter
17	LED Button 2	Windscreen Wiper
18	LED Button 3	Headlights
19	LED Button 4	Reset / centre menu

Note: After virtualization, these become vJoy buttons 33-51. The in-game mapping is identical in function; only the button numbers change.

10.7 LED Event Reference

LED	Colour	Trigger	SimHub NCALC Formula
LED1	Red	<= 3 laps fuel remaining	IF([Fuel]/[FuelPerLap] <= 3, 'LED1_FADE_FLASH_100', 'LED1_OFF')
LED2	Green	Tyre wear < 40%	IF([TyresWearMin] < 40, 'LED2_FADE_FLASH_100', 'LED2_OFF')

Table 23 – continued

LED	Colour	Trigger	SimHub NCALC Formula
LED3	Blue	Blue flag shown	IF([Flag_Blue], 'LED3_FADE_FLASH_100', 'LED3_OFF')
LED4	Yellow	Yellow flag shown	IF([Flag_Yellow], 'LED4_FADE_FLASH_100', 'LED4_OFF')
All 4	Chase	Best lap set	changed(ms, [BestLapTime]) with staggered delays

10.8 Serial Command Reference (Pico 2 Protocol)

Command	Effect
LED1_ON	LED1 full brightness instantly
LED1_OFF	LED1 to dim floor (1/255)
LED1_FLASH_300	LED1 harsh blink, 300ms on/off
LED1_FADE_ON	LED1 smooth fade to full brightness
LED1_FADE_OFF	LED1 smooth fade to dim floor
LED1_FADE_FLASH_100	LED1 smooth breathing fade, 100ms half-cycle
DISP Line1 Line2 Line3 Line4	Update OLED display (1-4 lines)

Replace LED1 with LED2, LED3, or LED4 as needed. All commands end with \n.

Appendix A: Complete Parts List (Shopping List)

Electronics

- Raspberry Pi Pico 2 x 1
- Freenove Terminal Breakout for Pico 2 x 1
- KY-040 Rotary Encoder (5-pin) x 3
- Mini (On)-Off-(On) Momentary Toggle Switch x 3
- 16mm Illuminated Momentary Pushbutton (4-pin) x 4
- 2.42" SSD1309 OLED I2C Display x 1
- 220-470 Ohm Resistors x 4
- 22AWG Solid-Core Wire (multi-colour) x 1 pack
- Micro-USB or USB-C Cable (data-capable) x 1

Enclosure

- 12mm MDF Sheet (810x405mm minimum) x 1
- 2mm Aluminium Sheet (200x150mm) x 1
- M2 Nylon Washers x 4
- M2/M3/M4 Screw Assortment x 1 set
- M3 or M4 Threaded Inserts (for faceplate) x 6

Tools

- 90W Digital Soldering Iron

- Solder (0.8mm)
- Wire Strippers
- Flush Cutters
- Needle-Nose Pliers
- Small Phillips Screwdriver
- Drill + 6mm, 7mm, 16mm bits
- Jigsaw or CNC Router
- 120-Grit Sandpaper
- Masking Tape + Marker
- Multimeter (optional but recommended)
- Helping Hands (optional)

Software (Free)

- Arduino IDE 2.x
- SimHub
- vJoy 2.1.9.1
- Joystick Gremlin R15 (original)
- HIDHide
- AutoHotkey v1.1

Appendix B: GPIO Pin Usage Summary

GPIO	Function	Component
GP0	Encoder 1 CLK	Encoder 1
GP1	Encoder 1 DT	Encoder 1
GP2	Encoder 1 SW	Encoder 1
GP3	Encoder 2 CLK	Encoder 2
GP4	Encoder 2 DT	Encoder 2
GP5	Encoder 2 SW	Encoder 2
GP6	Encoder 3 CLK	Encoder 3
GP7	Encoder 3 DT	Encoder 3
GP8	Encoder 3 SW	Encoder 3
GP9	SPARE	—
GP10	Toggle 1 Top	Toggle Switch 1
GP11	Toggle 1 Bottom	Toggle Switch 1
GP12	Toggle 2 Top	Toggle Switch 2
GP13	Toggle 2 Bottom	Toggle Switch 2
GP14	Toggle 3 Top	Toggle Switch 3
GP15	Toggle 3 Bottom	Toggle Switch 3
GP16	OLED SDA	OLED Screen
GP17	OLED SCL	OLED Screen
GP18	Button 1 Switch	LED Button 1
GP19	Button 2 Switch	LED Button 2
GP20	Button 3 Switch	LED Button 3
GP21	Button 4 Switch	LED Button 4

Table 25 – continued

GPIO	Function	Component
GP22	Button 1 LED+	LED Button 1
GP26	Button 2 LED+	LED Button 2
GP27	Button 3 LED+	LED Button 3
GP28	Button 4 LED+	LED Button 4

This guide documents the complete working setup as of August 2026. Do not modify components unless you understand the impact on the full chain (Pico 2 -> Gremlin -> vJoy -> HIDHide -> Game -> SimHub -> Pico 2).

Good luck, and happy racing.